Exploiting a Mass Assignment Vulnerability

Web Application Penetration Testing

Platform	PortSwigger Web Security Academy	**Tool**	Burp Suite Community Edition
Category	API Testing / Mass Assignment	**Status**	Completed

1 Objective

The objective was to identify whether the checkout API accepted a client-controlled field that affected the final price calculation, and to determine whether that field could be manipulated to complete an order without sufficient store credit.

2 Background

Mass assignment can occur when an application automatically maps fields supplied by a client into internal data objects without restricting which fields the client is allowed to set. If sensitive fields are accepted, a user may be able to change values that should be calculated or controlled by the server.

In this lab, the field of interest was `chosen_discount.percentage` in the checkout request.

3 Methodology

1. Signed in to the lab application and added the Lightweight l33t Leather Jacket to the basket.
2. Used Burp Suite HTTP history to inspect the API traffic generated by the basket and checkout functionality.
3. Sent a request to `GET /api/checkout` and examined the JSON response. It exposed `chosen_discount`, `chosen_products`, and the product's `item_price`.
4. Inspected the checkout request and identified that the JSON body included a `chosen_discount` object with a `percentage` field.
5. Tested the checkout request with the discount percentage set to `0`. The server returned a response indicating insufficient funds.
6. Modified the client-supplied discount percentage to `100` and resent the checkout request.
7. Observed a successful order-creation response and the order-confirmation location in the response headers.
8. Verified that the lab displayed the successful completion message.

4 Technical Evidence

4.1 Checkout JSON structure

The `GET /api/checkout` response showed the JSON fields used by the checkout process, including the selected discount and product details.



Figure 1. Checkout API response showing the JSON structure and product details.

4.2 Baseline request (discount = 0)

With the discount at `0`, the server answered `201 Created` but the `Location` header redirected to `/cart?err=INSUFFICIENT_FUNDS`, so no order was placed.

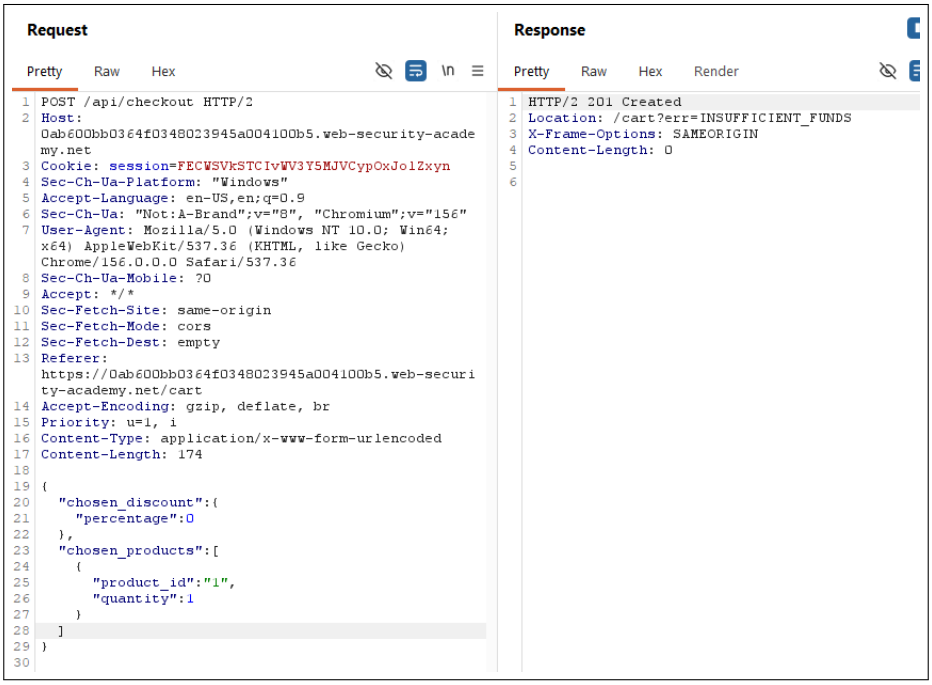


Figure 2. Baseline checkout request with `percentage` set to 0.

4.3 Modified request (discount = 100)

With `chosen_discount.percentage` changed to `100`, the server returned `201 Created` and a `Location` header pointing to the order-confirmation page.

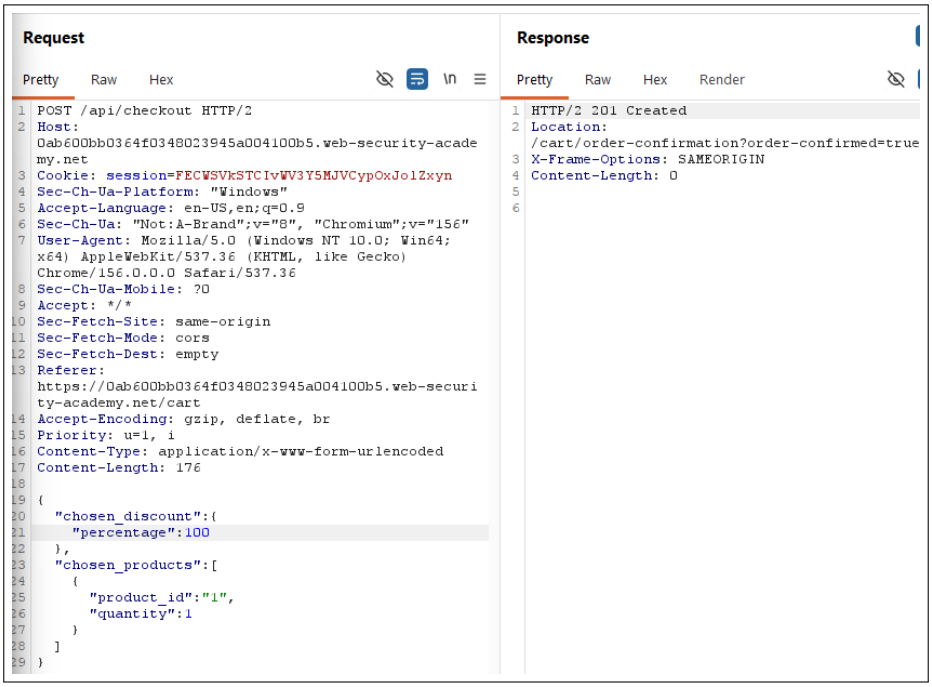


Figure 3. Successful checkout response after modifying the discount field to 100.

5 Result

The lab was successfully completed. The checkout API accepted a client-supplied discount percentage of 100 and created the order. The normal checkout attempt with a zero discount returned an insufficient-funds result. This supports the finding that the server trusted a client-controlled field that should have been determined or validated by the application.

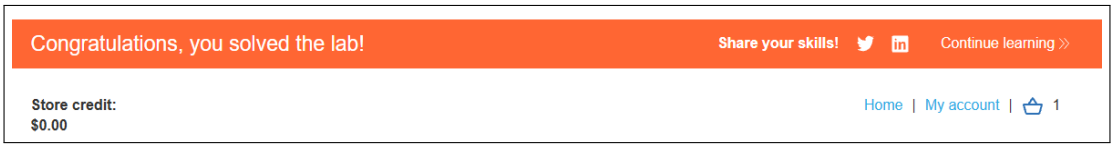


Figure 4. PortSwigger confirmation that the lab was solved.

6 Security Impact

In a real application, trusting client-supplied pricing or discount fields could allow users to reduce the amount charged, bypass payment checks, or obtain products or services without paying the intended price. The actual impact depends on how the application processes orders and payments.

7 Recommendations

- Calculate discounts and prices on the server using trusted product and promotion data.
- Do not accept sensitive fields such as discount percentages or final prices directly from the client.

- Use explicit allowlists of fields that each API operation is permitted to update.
- Validate checkout totals and payment amounts on the server immediately before creating an order.
- Add tests to confirm that unexpected or unauthorized JSON properties are rejected or ignored.

8 Conclusion

This lab demonstrated a mass assignment issue in the checkout API. By inspecting the JSON structure and modifying the `chosen_discount.percentage` field, the checkout request was accepted with a 100 percent discount. Sensitive pricing decisions should be calculated and enforced by the server rather than trusted from client-supplied request data.

All testing was performed in the authorized PortSwigger Web Security Academy lab environment.